

Как да напишем `virtual.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · `virtual`, абстрактен клас, два базови класа

Какво ще направим

В упражнение 2 открихме нещо, което тогава оставихме нерешено:

```
CPerson* pPerson = &stud3;    // stud3 е CStudent
pPerson->Output(cout);         // извежда само личностната част!
```

Обектът си беше студент, но се извикваше функцията на `CPerson`. Едната дума `virtual` оправя това — и оттам нататък целият код може да работи с базови указатели, без да знае какъв точно обект стои отдолу.

Изискване	Какво добавяме
1–3	<code>operator>></code> за <code>CAddress</code> , <code>CPerson</code> и <code>CStudent</code> — огледалото на <code><<</code>
4	абстрактен клас <code>CGSM</code> с три напълно виртуални функции
5	<code>CStudent</code> става наследник на два класа; добавя се телефонен номер
6	член-оператор <code>></code> за сравнение по тарифен план
7–8	главна функция и четене на масив от студенти от <code>Tema05.txt</code>

План на работата

- Нов проект и класовете от упражнение 4
- `operator>>` за `CAddress` (изискване 1)
- `virtual` — и защо операторите не могат да бъдат виртуални
- Виртуален деструктор
- Абстрактният клас `CGSM` (изискване 4)
- Множествено наследяване (изискване 5)
- Четене на студент (изисквания 2–3)
- Член-оператор `>` (изискване 6)
- Главна функция и четене от файл (изисквания 7–8)

1 Нов проект и класовете от упражнение 4

Нов **Empty Project**, например `Tema5_Virtual`, файл `virtual.cpp`. Пренесете `CAddress`, `CPerson` и `CStudent` от упражнение 4 заедно с операторите `<<` и сравненията по възраст.

Ще ви трябва и текстов файл `Tema05.txt` в папката на проекта:

```
Ivanov 1111111104
Varna 9010 Studentska#1
21621301 NOKIA 20 0888000000
Ivanka 1111111110
Varna 9010 Studentska#2
21621302 NOKIA 10 0888000001
Petrov 7503081239
Sofia 1000 Tsar_Osvoboditel#12
21621303 SAMSUNG 35 0888000002
Elena 0147222342
Burgas 8000 Aleksandrovska#5
21621304 APPLE 15 0888000003
```

Форматът от условието е три реда на студент:

```
<име> <ЕГН>
<Град> <Пощ. код> <Адрес>
<Факъл> <МаркаGSM> <ТарифенПлан> <Телъл>
```

Забележете, че в данните няма интервали вътре в стойностите — пише се `Studentska#1`, а не `Studentska #1`, и `Tsar_Osvoboditel#12` с долна черта. Причината е в стъпка 2: операторът `>>` спира при първия интервал. Ако улицата съдържа интервал, тя ще се прочете само наполовина, а остатъкът ще отиде в следващото поле и всичко след него ще се разпадне.

2 operator>> за CAddress

ОБЯВЕТЕ В КЛАСА И НАПИШЕТЕ СЛЕД НЕГО

```
friend istream& operator>>(istream& fromStream, CAddress& addr);
```

```
istream& operator>>(istream& fromStream, CAddress& addr)
{
    // Редът е точно като във файла: град, пощенски код, улица.
    fromStream >> addr.m_strCity >> addr.m_strPostCode >> addr.m_strStreet;
    return fromStream;
}
```

РАЗЛИКИТЕ СПРЯМО OPERATOR<<

	operator<<	operator>>
Поток	ostream& — изходен	istream& — входен
Обект	const CAddress&	CAddress& — без const
Посока	чете от обекта	пише в обекта

Липсата на const при >> е задължителна. Функцията променя обекта — това ѝ е работата. Ако напишете `const CAddress&`, всеки ред вътре ще даде `C3892: cannot assign to a variable that is const`.

КАК >> РАЗДЕЛЯ ДАННИТЕ

Операторът `>>` пропуска всички празни знаци отпред, чете до следващия празен знак и спира. „Празен знак“ е интервал, табулация и нов ред — за него те са едно и също.

Оттам следва нещо важно: трите реда на студент във файла не са три реда за програмата. Тя вижда поредица от думи. Затова не е нужно нищо да брои редове — просто се четат полетата в реда, в който са записани. Ако форматът изискваше нещо друго (например улица с интервали), щеше да се наложи `getline`.

Проверка: добавете временно в `main` :

```
istringstream iss("Varna 9010 Studentska#1");
CAddress addr;
iss >> addr;
cout << addr << endl;
```

Трябва да изведе `Varna 9010, Studentska#1`. `istringstream` е поток, който чете от низ — удобен за проверка, без да пипате файлове. Нужен е `#include <sstream>`.

3 virtual — сърцевината на темата

ПЪРВО ПРОБЛЕМЪТ, ОЩЕ ВЕДНЪЖ

Без `virtual` компилаторът избира функцията по **типа на указателя**, още при компилацията. Нарича се *ранно свързване* (early binding). Указателят е `CPerson*`, значи се вика `CPerson::Output` — независимо че обектът е студент.

С `virtual` изборът се отлага за момента на изпълнението и се прави по **типа на обекта**. Нарича се *късно свързване* (late binding).

ДОБАВЕТЕ VIRTUAL В CPERSON

```
// ВИРТУАЛНИ функции за вход/изход – наследникът ги подменя
virtual ostream& Output(ostream& toStream) const;
virtual istream& Input(istream& fromStream);
```

И В CSTUDENT

```
virtual ostream& Output(ostream& toStream) const;
virtual istream& Input(istream& fromStream);
```

Как работи отвътре. Клас с виртуални функции получава скрита таблица с адреси на функции (*виртуална таблица*), а всеки обект — скрит указател към нея. При `pPerson->Output(cout)` програмата поглежда таблицата на конкретния обект и вика адреса оттам. Затова обектът става с 8 байта по-голям, а извикването — с едно четене от паметта по-бавно. За сметка на това кодът може да работи с базови указатели, без да знае типа.

КЛЮЧОВАТА ДУМА `VIRTUAL` СЕ ПИШЕ ВЕДНЪЖ

- 1 Щом функцията е `virtual` в базовия клас, тя е виртуална и във **всички** наследници — дори да не напишете думата. Пише се пак за четимост.
- 2 **Сигнатурата трябва да съпада точно** — име, типове на параметрите и `const`. При разминаване не получавате грешка, а *нова* функция, която скрива старата. Тогава всичко се компилира и пак работи грешно.
- 3 C++11 дава защита: напишете `override` след сигнатурата в наследника и при разминаване компилаторът ще откаже с `C3668`. Силно препоръчително.

НО ОПЕРАТОРЪТ `<<` НЕ МОЖЕ ДА БЪДЕ ВИРТУАЛЕН

От упражнение 4 знаем, че `operator<<` е **глобална** функция. Виртуални могат да бъдат само член-функции. Как тогава `cout << *pPerson` да изведе целия студент?

Решението е стандартният начин в C++: операторът остава един, а работата се прехвърля на виртуална функция.

```
ostream& operator<<(ostream& toStream, const CPerson& pers)
{
    return pers.Output(toStream);    // ТУК се случва виртуалното извикване
}

istream& operator>>(istream& fromStream, CPerson& pers)
{
    return pers.Input(fromStream);
}
```

Два реда, които заменят цялата конструкция от упражнение 4. Операторът приема `const CPerson&`. Понеже студентът е личност, при `cout << stud` се вика точно този оператор. Вътре `pers.Output(...)` е виртуално извикване и отива в `CStudent::Output`.

Следствие: отделният `operator<<` за `CStudent` от упражнение 4 **вече не е нужен** — премахнете го. Един оператор обслужва целия клон на наследяване, включително класове, които още не са написани.

`CAddress` си запазва собствен оператор — тя не е част от тази йерархия.

Вътре в `CStudent::Output` внимавайте с наследената част. Пише се `CPerson::Output(toStream);` — с изричното `CPerson::`. То изключва виртуалния механизъм за това конкретно извикване и вика точно версията на базата. Без него функцията вика себе си: безкрайна рекурсия и *Stack overflow*.

4 Виртуален деструктор

Има още едно място, където липсата на `virtual` мълчаливо чупи програмата.

```
CPerson* pDyn = new CStudent(...);
delete pDyn;    // кой деструктор се вика?
```

Без `virtual` се вика `~CPerson`. Деструкторът на `CStudent` не се изпълнява — а именно той освобождава масива с оценки. Резултат: теч на памет при всяко изтриване.

ДОБАВЕТЕ В CPERSON И CGSM

```
virtual ~CPerson();
```

Правилото: ако класът има поне една виртуална функция, деструкторът му също трябва да е виртуален. Практически това значи: *ако смятате да наследявате от класа и да го изтривате през базов указател, деструкторът е virtual*. Цената е нула, ако виртуална таблица и без това има.

Обратното също важи: клас, който не е предвиден за наследяване, няма нужда от виртуален деструктор.

Компиляторът **не предупреждава** за липсващ виртуален деструктор при стандартните настройки. Грешката се вижда само като бавно нарастваща заета памет.

5 Абстрактният клас CGSM

Условието иска клас с две частни променливи, два конструктора и **три напълно виртуални** функции.

ДОБАВЕТЕ ПРЕДИ CSTUDENT

```
class CGSM
{
private:
    string m_strModel;        // модел, напр. "NOKIA"
    int    m_nTariff;         // тарифен план, напр. 20

public:
    CGSM();                  // подразбиращ се
    CGSM(const string& strModel, int nTariff);    // експлицитен
    virtual ~CGSM();         // виртуален деструктор

    const string& GetModel() const;
    int    GetTariff() const;
    void SetModel(const string& strModel);
    void SetTariff(int nTariff);

    // НАПЪЛНО ВИРТУАЛНИ функции: = 0 значи "няма тяло тук".
    // Всеки наследник е длъжен да ги напише.
    virtual ostream& OutputGSM(ostream& toStream) const = 0;
    virtual string  getGSMNumber() const = 0;
    virtual istream& InputGSM(istream& fromStream) = 0;
};
```

КАКВО ЗНАЧИ = 0

- 1 **Функцията няма тяло в този клас.** `= 0` не значи „връща нула“ — това е синтаксис, който казва „тук няма реализация“.
- 2 **Класът става абстрактен.** Не може да се създаде обект от него: `CGSM gsm;` дава `C2259: cannot instantiate abstract class`. Може обаче да има указател `CGSM*` и референция `CGSM&` — и точно за това служи.

- 3 **Наследникът е длъжен да напише и трите.** Ако пропусне една, той също остава абстрактен и `CStudent` `stud`; няма да се компилира — със същата грешка `C2259`.

Защо е удобно да се напише абстрактен клас. `CGSM` е *договор*: който наследява този клас, гарантира, че има трите функции. Кодът, който работи с `CGSM*`, може да ги вика, без да знае нищо за конкретния клас. Утре някой напише `CEmployee` : `public CGSM` и същият код ще работи и с него.

ЧАСТНИ ПРОМЕНЛИВИ И НАСЛЕДНИК

Условието изисква `m_strModel` и `m_nTariff` да са **частни**. Частното не се вижда от наследника — значи `CStudent` не може да пише `m_nTariff` направо. Затова `CGSM` предлага `GetModel`, `GetTariff`, `SetModel` и `SetTariff`, а наследникът минава през тях.

Другата възможност е `protected`: вместо `private`: — както направихме с `CPerson` в упражнение 2. Тук избираме частни членове с достъп през функции, защото условието изрично казва „частни“, а и резултатът е по-добрият: базовият клас пази контрол върху данните си.

6 Множествено наследяване

ПРОМЕНЕТЕ ЗАГЛАВНИЯ РЕД НА CStudent

```
class CStudent : public CPerson, public CGSM
{
private:
    string    m_strFN;
    CAddress  m_address;
    string    m_strPhone;    // 5. номер на телефон
    ...
}
```

Един студент сега е три неща едновременно: личност (`CPerson`), притежател на телефон (`CGSM`) и студент със собствени данни.

ВСИЧКИ КОНСТРУКТОРИ ТРЯБВА ДА ВИКАТ ДВАТА БАЗОВИ КЛАСА

```
CStudent::CStudent()
    : CPerson(), CGSM(), m_strFN("(няма)"), m_address(), m_strPhone("(няма)"),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

// Нов конструктор – с всички данни наведнъж
CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
                  const string& strFN, const string& strModel, int nTariff,
                  const string& strPhone)
    : CPerson(strName, "неопределен", strEGN), CGSM(strModel, nTariff),
      m_strFN(strFN), m_address(addr), m_strPhone(strPhone),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}
```

И КОПИРАЩИЯТ КОНСТРУКТОР, И ОПЕРАТОРЪТ =

```
CStudent::CStudent(const CStudent& stud)
    : CPerson(stud), CGSM(stud),          // ДВАТА базови класа
      m_strFN(stud.m_strFN), m_address(stud.m_address), m_strPhone(stud.m_strPhone),
```

```

...

CStudent& CStudent::operator=(const CStudent& stud)
{
    if (this == &stud) return *this;

    CPerson::operator=(stud);          // базов клас 1
    CGSM::operator=(stud);             // базов клас 2
    ...
}

```

Забравен втори базов клас е грешка, която не личи. Ако в `operator=` пропуснете `CGSM::operator=(stud);`, всичко ще се компилира, но моделът и тарифният план няма да се копират — те ще си останат от лявата страна. Проверката е проста: след `a = b;` сравнете `a.GetTariff()` с `b.GetTariff()`.

РЕДЪТ НА ПОСТРОЯВАНЕ

Базовите класове се построяват в реда, в който са изброени в заглавния ред на класа — първо `CPerson`, после `CGSM` — независимо от реда в списъка за инициализация. Унищожаването е в обратен ред.

ТРИТЕ ЗАДЪЛЖИТЕЛНИ ФУНКЦИИ

```

ostream& CStudent::OutputGSM(ostream& toStream) const
{
    toStream << GetModel() << " (план " << GetTariff() << ", тел. "
        << getGSMNumber() << ")";
    return toStream;
}

string CStudent::getGSMNumber() const { return m_strPhone; }

istream& CStudent::InputGSM(istream& fromStream)
{
    string strModel, strPhone;
    int nTariff = 0;
    if (fromStream >> strModel >> nTariff >> strPhone)
    {
        SetModel(strModel);
        SetTariff(nTariff);
        m_strPhone = strPhone;
    }
    return fromStream;
}

```

Защо номерът е в `CStudent`, а не в `CGSM`. Условието нарочно го иска така: `CGSM` описва *апарата* (модел, тарифа), а номерът е на *притежателя*. Затова `getGSMNumber` е напълно виртуална в `CGSM` — базовият клас знае, че номер има, но не знае откъде идва. Точно това е смисълът на напълно виртуалната функция.

Внимание при множествено наследяване. Ако двата базови класа имат функция с еднакво име, обръщението става двусмислено (C2385: `ambiguous access`) и трябва да се пише `stud.CGSM::Output(...)`. Ако пък двата базови класа наследяват общ трети, той се появява **два пъти** в обекта — решава се с виртуално наследяване (`class B : virtual public A`). Тук нищо такова няма: `CPerson` и `CGSM` са напълно независими и точно затова примерът е безопасен.

7 Четене на личност и на студент

CPerson::Input

```
istream& CPerson::Input(istream& fromStream)
{
    // Формат: <име> <ЕГН>
    string strName, strEGN;
    if (fromStream >> strName >> strEGN)
    {
        m_strName = strName;
        SetEGN(strEGN);           // невалидно ЕГН просто не се записва
    }
    return fromStream;
}
```

Защо първо в локални променливи? Ако четенето се провали по средата (край на файла, повредени данни), обектът остава с досегашните си стойности, вместо да стане наполовина попълнен. Записваме едва след като целият ред е прочетен успешно.

CStudent::Input – ТРИТЕ РЕДА НАВЕДНЪЖ

```
istream& CStudent::Input(istream& fromStream)
{
    CPerson::Input(fromStream);    // ред 1: име и ЕГН
    fromStream >> m_address;        // ред 2: град, код, улица
    fromStream >> m_strFN;          // ред 3, първа част
    InputGSM(fromStream);          // ред 3, останалото
    return fromStream;
}

// Операторът само делегира:
istream& operator>>(istream& fromStream, CStudent& stud)
{
    return stud.Input(fromStream);
}
```

Четири реда, всеки от които използва вече написана функция: базовия клас, оператора на `CAddress` и виртуалната `InputGSM`.

8 Член-оператор > по тарифен план

Условието задава: `bool operator>(const CStudent& studR) const` — и изрично уточнява, че **не** е глобална приятелска функция.

ОБЯВЕТЕ В КЛАСА И НАПИШЕТЕ СЛЕД НЕГО

```
bool operator>(const CStudent& studR) const;
```

```
// Само ЕДИН параметър: лявата страна е самият обект (this).
bool CStudent::operator>(const CStudent& studR) const
```

```
{
    return GetTariff() > studR.GetTariff();
}
```

ЧЛЕН СРЕЩУ ГЛОБАЛНА ФУНКЦИЯ

	Глобален < (упр. 4)	Член > (тук)
Брой параметри	два	един — левият е <code>this</code>
Как се пише	<code>bool operator<(const CStudent&, const CStudent&)</code>	<code>bool CStudent::operator>(const CStudent&) const</code>
Достъп до частни	нужен е <code>friend</code>	има го по рождение
<code>const</code> накрая	няма къде	задължително — иначе не работи с константни обекти

Най-честата грешка: член-операторът се пише с два параметъра. Тогава Visual Studio дава C2804: binary 'operator >' has too many parameters. Като член левият операнд е самият обект и не се изброява.

Върху едни и същи два обекта сега има две различни подредби. `a < b` сравнява по възраст, `a > b` — по тарифен план. Не са противоположни и не бива да се очаква да са. Това е напълно законно, но е добър пример защо операторите се предефинират пестеливо: от `a > b` никой не може да отгатне, че става дума за телефонен план. Затова в програмата изрично пишем какво се сравнява на всеки ред.

9 Главна функция и четене от файл

9.1 Виртуалните функции на дело

```
CStudent studDemo("Георги Стоянов", addrVarna, "7503081239", "21621301",
                  "NOKIA", 20, "0888000000");
studDemo.SetGender("мъж");
studDemo.AddGrade(5.50);

CPerson* pPerson = &studDemo;           // указател към БА30В клас
cout << " през CStudent&: " << studDemo << endl;
cout << " през CPerson*: " << *pPerson << endl;
```

Проверка: двата реда трябва да са **еднакви**. Сравнете с упражнение 2, където вторият показваше само личностната част. Ако и сега е така — липсва `virtual` пред `Output` в `CPerson`.

9.2 Абстрактният клас през указател

```
CGSM* pGSM = &studDemo;                 // студентът Е и CGSM
cout << " през CGSM* -> OutputGSM: ";
pGSM->OutputGSM(cout) << endl;
```

```
cout << " през CGSM* -> getGSMNumber: " << pGSM->getGSMNumber() << endl;
// CGSM gsm;          // грешка: абстрактен клас не може да се създаде
```

Задача за проверка: разкоментирайте последния ред. Очаква се C2259: 'CGSM': cannot instantiate abstract class , придружено от списък на функциите, които липсват. Върнете реда като коментар.

9.3 Четене на масив от студенти

```
const int MAX_STUD = 10;
CStudent arrStud[MAX_STUD];
int nRead = 0;

fstream iFile("Tema05.txt", ios_base::in);
if (!iFile)
{
    cout << " Файлът Tema05.txt не е намерен!" << endl;
}
else
{
    while (nRead < MAX_STUD && (iFile >> arrStud[nRead]))
        nRead++;
    iFile.close();
}
```

- 1 `(iFile >> arrStud[nRead])` е едновременно четене и проверка. Операторът връща потока, а потокът в логически контекст дава „лъжа“ при край на файла или при грешка. Затова цикълът спира сам.
- 2 `nRead < MAX_STUD` стои **първо**. Ако файлът съдържа повече от 10 студента, без тази проверка ще се пише извън масива. Редът има значение: `&&` не изчислява дясната страна, ако лявата е лъжа.
- 3 `CStudent arrStud[MAX_STUD];` изисква конструктор по подразбиране — затова той не бива да се пропуска, дори когато изглежда безполезен.

Къде да сложите Tema05.txt . В папката с файла `.vcxproj` , не в `Debug` . Ако програмата казва, че не го намира, добавете временно:

```
#include <filesystem>
cout << std::filesystem::current_path() << endl;
```

и вижте откъде точно търси.

9.4 Тест на двете сравнения

```
cout << " по ВЪЗРАСТ (глобален <): " << (arrStud[0] < arrStud[2] ? "да" : "не") << endl;
cout << " по ТАРИФА (член >): " << (arrStud[0] > arrStud[2] ? "да" : "не") << endl;

// Най-скъпият тарифен план
int nMax = 0;
for (int i = 1; i < nRead; i++)
    if (arrStud[i] > arrStud[nMax])
        nMax = i;
```

9.5 Масив от указатели към базовия клас

```
CPerson persPlain("Иван Петров", "мъж", "8512034566");
CPerson* arrPtr[3] = { &persPlain, &studDemo, &arrStud[0] };
for (int i = 0; i < 3; i++)
    cout << "  [" << i << "] " << *arrPtr[i] << endl;
```

Това е целта на цялата тема. Масивът е от указатели към `CPerson`, но съдържа и личност, и двама студенти. Един и същи ред код извежда различно съдържание за всеки — защото `Output` е виртуална. Цикълът не знае и не му трябва да знае какви обекти обхожда.

Разгледайте изхода: ред `[0]` е къс (само личност), а `[1]` и `[2]` са пълни студентски записи.

9.6 Виртуалният деструктор

```
CPerson* pDyn = new CStudent("Петър Колев", addrVarna, "0147222342", "21621399",
                             "APPLE", 15, "0888000009");
cout << "  " << *pDyn << endl;
delete pDyn;          // вика ~CStudent, защото деструкторът е virtual
```

Задача за проверка: махнете временно `virtual` пред `~CPerson()` и поставете точка на прекъсване (`F9`) в `~CStudent`. Стартирайте с `F5` — при `delete pDyn` прекъсването **няма** да се задейства. Върнете `virtual` и опитайте пак.

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. Работната папка е тази на проекта — оттам се търси <code>Tema05.txt</code> .
<code>F11</code>	Step Into. Влезте в <code>*pPerson</code> извеждането и вижте как се стига до <code>CStudent::Output</code> .

```
cl /EHsc /std:c++17 /utf-8 virtual.cpp
virtual.exe
```

Кирилица под Windows

Файлът се записва като *Unicode (UTF-8 with signature)* — `Codepage 65001`; компилаторът получава `/utf-8`; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main`.

Данните в `Tema05.txt` са нарочно на латиница. Така файлът се чете еднакво независимо от кодировката му. Ако сложите кирилица вътре, запишете го в UTF-8 — иначе имената ще се прочетат като безсмислени символи.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 5: Виртуални функции, множествено наследяване.
// Надгражда класовете от упражнение 4: добавят се оператори за четене от
// поток, абстрактен клас CGSM и CStudent става наследник на ДВА класа.

#include <cctype>      // isdigit
#include <cstdio>      // getchar
#include <fstream>     // fstream
#include <iomanip>      // setw, setfill
#include <iostream>
#include <sstream>      // ostringstream
#include <string>
using namespace std;

enum Gender
{
    MALE,           // мъж
    FEMALE,         // жена
    UNKNOWN         // неопределен
};

// =====
// CAddress
// =====
class CAddress
{
private:
    string m_strStreet;
    string m_strPostCode;
    string m_strCity;

public:
    CAddress();
    CAddress(const string& strStreet, const string& strPostCode, const string& strCity);
    CAddress(const CAddress& addr);

    const string& GetStreet() const;
    const string& GetPostCode() const;
    const string& GetCity() const;
    void SetStreet(const string& strStreet);
    void SetPostCode(const string& strPostCode);
    void SetCity(const string& strCity);

    friend ostream& operator<<(ostream& toStream, const CAddress& addr);
    // 1. Оператор за ЧЕТЕНЕ: формат <Град> <Пощ.код> <Адрес>
    friend istream& operator>>(istream& fromStream, CAddress& addr);
};

CAddress::CAddress()
    : m_strStreet("(няма улица)", m_strPostCode("0000"), m_strCity("(няма град)") {}

CAddress::CAddress(const string& strStreet, const string& strPostCode, const string& strCity)
    : m_strStreet(strStreet), m_strPostCode(strPostCode), m_strCity(strCity) {}
```

```

CAddress::CAddress(const CAddress& addr)
    : m_strStreet(addr.m_strStreet), m_strPostCode(addr.m_strPostCode),
      m_strCity(addr.m_strCity) {}

const string& CAddress::GetStreet() const { return m_strStreet; }
const string& CAddress::GetPostCode() const { return m_strPostCode; }
const string& CAddress::GetCity() const { return m_strCity; }

void CAddress::SetStreet(const string& strStreet) { m_strStreet = strStreet; }
void CAddress::SetPostCode(const string& strPostCode) { m_strPostCode = strPostCode; }
void CAddress::SetCity(const string& strCity) { m_strCity = strCity; }

ostream& operator<<(ostream& toStream, const CAddress& addr)
{
    toStream << addr.m_strCity << " " << addr.m_strPostCode << ", " << addr.m_strStreet;
    return toStream;
}

// ----- 1. Четене на адрес -----
istream& operator>>(istream& fromStream, CAddress& addr)
{
    // Редът е точно като във файла: град, пощенски код, улица.
    fromStream >> addr.m_strCity >> addr.m_strPostCode >> addr.m_strStreet;
    return fromStream;
}

// =====
// CPerson – вече с ВИРТУАЛНИ функции
// =====
class CPerson
{
protected:
    string m_strName;
    Gender m_gender;
    string m_strEGN;

private:
    void getYMD(const string& strEGN, int& iYear, int& iMonth, int& iDay) const;

public:
    CPerson();
    CPerson(const string& strName, const string& strGender, const string& strEGN);
    CPerson(const CPerson& pers);

    // Виртуален деструктор – задължителен при наследяване с указатели
    virtual ~CPerson();

    void SetName(const string& strName);
    void SetGender(Gender gender);
    void SetGender(const string& strGender);
    bool SetEGN(const string& strEGN);

    const string& GetName() const;
    Gender GetGender() const;
    const string& GetEGN() const;

    Gender stringToGender(const string& strGender) const;
    string genderToString() const;

```

```

bool isValid(const string& strEGN);

int GetBirthKey() const;
string GetBirthDate() const;

// ВИРТУАЛНИ функции за вход/изход – наследникът ги подменя
virtual ostream& Output(ostream& toStream) const;
virtual istream& Input(istream& fromStream);

friend ostream& operator<<(ostream& toStream, const CPerson& pers);
// 2. Оператор за четене на личност
friend istream& operator>>(istream& fromStream, CPerson& pers);
};

CPerson::CPerson()
    : m_strName("Неизвестен"), m_gender(UNKNOWN), m_strEGN("") {}

CPerson::CPerson(const string& strName, const string& strGender, const string& strEGN)
    : m_strName(strName), m_gender(UNKNOWN), m_strEGN("")
{
    m_gender = stringToGender(strGender);
    SetEGN(strEGN);
}

CPerson::CPerson(const CPerson& pers)
    : m_strName(pers.m_strName), m_gender(pers.m_gender), m_strEGN(pers.m_strEGN) {}

CPerson::~CPerson() {}

void CPerson::SetName(const string& strName)    { m_strName = strName; }
void CPerson::SetGender(Gender gender)         { m_gender = gender; }
void CPerson::SetGender(const string& strGender) { m_gender = stringToGender(strGender); }

bool CPerson::SetEGN(const string& strEGN)
{
    if (!isValid(strEGN)) return false;
    m_strEGN = strEGN;
    return true;
}

const string& CPerson::GetName() const { return m_strName; }
Gender CPerson::GetGender() const { return m_gender; }
const string& CPerson::GetEGN() const { return m_strEGN; }

Gender CPerson::stringToGender(const string& strGender) const
{
    if (strGender == "мъж" || strGender == "м" || strGender == "male") return MALE;
    if (strGender == "жена" || strGender == "ж" || strGender == "female") return FEMALE;
    return UNKNOWN;
}

string CPerson::genderToString() const
{
    switch (m_gender)
    {
        case MALE: return "мъж";
        case FEMALE: return "жена";
        default: return "неопределен";
    }
}

```

```

}

void CPerson::getYMD(const string& strEGN, int& iYear, int& iMonth, int& iDay) const
{
    iYear = iMonth = iDay = 0;
    if (strEGN.length() != 10) return;

    int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
    int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
    int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

    if (nMonth >= 1 && nMonth <= 12) nYear += 1900;
    else if (nMonth >= 21 && nMonth <= 32) { nYear += 1800; nMonth -= 20; }
    else if (nMonth >= 41 && nMonth <= 52) { nYear += 2000; nMonth -= 40; }
    else return;

    iYear = nYear; iMonth = nMonth; iDay = nDay;
}

bool CPerson::isValid(const string& strEGN)
{
    if (strEGN.length() != 10) return false;
    for (size_t i = 0; i < strEGN.length(); i++)
        if (!isdigit(static_cast<unsigned char>(strEGN[i]))) return false;

    int nYear, nMonth, nDay;
    getYMD(strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return false;

    int arrDays[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    bool bLeap = (nYear % 4 == 0 && nYear % 100 != 0) || (nYear % 400 == 0);
    if (nMonth == 2 && bLeap) arrDays[1] = 29;
    if (nDay < 1 || nDay > arrDays[nMonth - 1]) return false;

    const int arrWeights[9] = { 2, 4, 8, 5, 10, 9, 7, 3, 6 };
    int nSum = 0;
    for (int i = 0; i < 9; i++) nSum += (strEGN[i] - '0') * arrWeights[i];

    int nCheck = nSum % 11;
    if (nCheck == 10) nCheck = 0;
    return nCheck == (strEGN[9] - '0');
}

int CPerson::GetBirthKey() const
{
    int nYear, nMonth, nDay;
    getYMD(m_strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return 0;
    return nYear * 10000 + nMonth * 100 + nDay;
}

string CPerson::GetBirthDate() const
{
    int nYear, nMonth, nDay;
    getYMD(m_strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return "(няма)";

    ostringstream oss;
    oss << setfill('0') << setw(2) << nDay << "."

```

```

        << setw(2) << nMonth << "."
        << setw(4) << nYear;

    return oss.str();
}

// ----- Виртуалните функции за вход/изход -----
ostream& CPerson::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << genderToString() << "), ЕГН: "
        << (m_strEGN.empty() ? string("(няма)") : m_strEGN)
        << ", роден: " << GetBirthDate();
    return toStream;
}

istream& CPerson::Input(istream& fromStream)
{
    // Формат: <име> <ЕГН>
    string strName, strEGN;
    if (fromStream >> strName >> strEGN)
    {
        m_strName = strName;
        SetEGN(strEGN);          // невалидно ЕГН просто не се записва
    }
    return fromStream;
}

// ----- Операторите просто делегират на виртуалните функции -----
ostream& operator<<(ostream& toStream, const CPerson& pers)
{
    return pers.Output(toStream);    // ТУК се случва виртуалното извикване
}

istream& operator>>(istream& fromStream, CPerson& pers)
{
    return pers.Input(fromStream);
}

// =====
// 4. Абстрактен клас CGSM
// =====
class CGSM
{
    // Частни член променливи – наследникът стига до тях през функциите по-долу.
private:
    string m_strModel;    // модел, напр. "NOKIA"
    int    m_nTariff;     // тарифен план, напр. 20

public:
    CGSM();                // подразбира се
    CGSM(const string& strModel, int nTariff);    // експлицитен
    virtual ~CGSM();       // виртуален деструктор

    const string& GetModel() const;
    int    GetTariff() const;
    void SetModel(const string& strModel);
    void SetTariff(int nTariff);

    // НАПЪЛНО ВИРТУАЛНИ функции: = 0 значи "няма тяло тук".
    // Всеки наследник е длъжен да ги напише.

```

```

    virtual ostream& OutputGSM(ostream& toStream) const = 0;
    virtual string  getGSMNumber() const = 0;
    virtual istream& InputGSM(istream& fromStream) = 0;
};

CGSM::CGSM() : m_strModel("(няма)", m_nTariff(0) {}

CGSM::CGSM(const string& strModel, int nTariff)
    : m_strModel(strModel), m_nTariff(nTariff) {}

CGSM::~~CGSM() {}

const string& CGSM::GetModel() const { return m_strModel; }
int CGSM::GetTariff() const          { return m_nTariff; }
void CGSM::SetModel(const string& strModel) { m_strModel = strModel; }
void CGSM::SetTariff(int nTariff)          { m_nTariff = nTariff; }

// =====
// 5. CStudent – наследник на ДВА класа
// =====
class CStudent : public CPerson, public CGSM
{
private:
    string  m_strFN;
    CAddress m_address;
    string  m_strPhone;    // 5. номер на телефон

    double* m_pdGrades;
    size_t  m_nCount;
    size_t  m_nCapacity;

    void grow();

public:
    CStudent();
    CStudent(const string& strName, const CAddress& addr, const string& strEGN);
    CStudent(const string& strName, const CAddress& addr, const string& strEGN,
              const string& strFN);
    CStudent(const string& strName, const CAddress& addr, const string& strEGN,
              const string& strFN, const string& strModel, int nTariff,
              const string& strPhone);
    CStudent(const CStudent& stud);
    CStudent& operator=(const CStudent& stud);
    virtual ~CStudent();

    void  AddGrade(double dGrade);
    double GetGrade(size_t nIndex) const;
    size_t GetGradeCount() const;
    double GetAverage() const;

    const string&  GetFN() const;
    const CAddress& GetAddress() const;
    const string&  GetPhone() const;
    void SetFN(const string& strFN);
    void SetAddress(const CAddress& addr);
    void SetPhone(const string& strPhone);

    // Подменяме виртуалните функции на CPerson
    virtual ostream& Output(ostream& toStream) const;

```

```

virtual istream& Input(istream& fromStream);

// 4. Задължителните три функции на CGSM
virtual ostream& OutputGSM(ostream& toStream) const;
virtual string  getGSMNumber() const;
virtual istream& InputGSM(istream& fromStream);

// 6. ЧЛЕН-оператор (не приятелски) за сравнение по тарифен план
bool operator>(const CStudent& studR) const;

friend void ConsoleInputEGN(CStudent& stud);
// 3. Оператор за четене на студент
friend istream& operator>>(istream& fromStream, CStudent& stud);

// Оператори за сравнение по възраст от упражнение 4
friend bool operator==(const CStudent& studL, const CStudent& studR);
friend bool operator!=(const CStudent& studL, const CStudent& studR);
friend bool operator< (const CStudent& studL, const CStudent& studR);
friend bool operator>=(const CStudent& studL, const CStudent& studR);
};

void CStudent::grow()
{
    size_t nNew = (m_nCapacity == 0) ? 4 : m_nCapacity * 2;
    double* pdNew = new double[nNew];
    for (size_t i = 0; i < m_nCount; i++) pdNew[i] = m_pdGrades[i];
    delete[] m_pdGrades;
    m_pdGrades = pdNew;
    m_nCapacity = nNew;
}

CStudent::CStudent()
    : CPerson(), CGSM(), m_strFN("(няма)", m_address(), m_strPhone("(няма)"),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN)
    : CPerson(strName, "неопределен", strEGN), CGSM(),
      m_strFN("(няма)", m_address(addr), m_strPhone("(няма)"),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
                  const string& strFN)
    : CPerson(strName, "неопределен", strEGN), CGSM(),
      m_strFN(strFN), m_address(addr), m_strPhone("(няма)"),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
                  const string& strFN, const string& strModel, int nTariff,
                  const string& strPhone)
    : CPerson(strName, "неопределен", strEGN), CGSM(strModel, nTariff),
      m_strFN(strFN), m_address(addr), m_strPhone(strPhone),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

// Копиращият конструктор вика копиращите конструктори на ДВАТА базови класа.
CStudent::CStudent(const CStudent& stud)
    : CPerson(stud), CGSM(stud),
      m_strFN(stud.m_strFN), m_address(stud.m_address), m_strPhone(stud.m_strPhone),
      m_pdGrades(nullptr), m_nCount(stud.m_nCount), m_nCapacity(stud.m_nCapacity)
{

```

```

    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity];
        for (size_t i = 0; i < m_nCount; i++) m_pdGrades[i] = stud.m_pdGrades[i];
    }
}

CStudent& CStudent::operator=(const CStudent& stud)
{
    if (this == &stud) return *this;

    CPerson::operator=(stud);          // базов клас 1
    CGSM::operator=(stud);             // базов клас 2
    m_strFN    = stud.m_strFN;
    m_address  = stud.m_address;
    m_strPhone = stud.m_strPhone;

    delete[] m_pdGrades;
    m_pdGrades = nullptr;

    m_nCount    = stud.m_nCount;
    m_nCapacity = stud.m_nCapacity;
    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity];
        for (size_t i = 0; i < m_nCount; i++) m_pdGrades[i] = stud.m_pdGrades[i];
    }
    return *this;
}

CStudent::~~CStudent()
{
    delete[] m_pdGrades;
    m_pdGrades = nullptr;
}

void CStudent::AddGrade(double dGrade)
{
    if (dGrade < 2.0 || dGrade > 6.0) return;
    if (m_nCount == m_nCapacity) grow();
    m_pdGrades[m_nCount] = dGrade;
    m_nCount++;
}

double CStudent::GetGrade(size_t nIndex) const
{
    if (nIndex >= m_nCount) return 0.0;
    return m_pdGrades[nIndex];
}

size_t CStudent::GetGradeCount() const { return m_nCount; }

double CStudent::GetAverage() const
{
    if (m_nCount == 0) return 0.0;
    double dSum = 0.0;
    for (size_t i = 0; i < m_nCount; i++) dSum += m_pdGrades[i];
    return dSum / m_nCount;
}

```

```

const string& CStudent::GetFN() const { return m_strFN; }
const CAddress& CStudent::GetAddress() const { return m_address; }
const string& CStudent::GetPhone() const { return m_strPhone; }

void CStudent::SetFN(const string& strFN) { m_strFN = strFN; }
void CStudent::SetAddress(const CAddress& addr) { m_address = addr; }
void CStudent::SetPhone(const string& strPhone) { m_strPhone = strPhone; }

// ----- Подменените виртуални функции на CPerson -----
ostream& CStudent::Output(ostream& toStream) const
{
    CPerson::Output(toStream); // наследената част
    toStream << ", ф.н.: " << m_strFN << ", адрес: " << m_address;

    toStream << ", GSM: ";
    OutputGSM(toStream); // частта от CGSM

    toStream << ", оценки: ";
    if (m_nCount == 0)
        toStream << "(няма)";
    else
    {
        for (size_t i = 0; i < m_nCount; i++)
            toStream << m_pdGrades[i] << (i + 1 < m_nCount ? " " : "");
        toStream << " | ср. успех: " << GetAverage();
    }
    return toStream;
}

istream& CStudent::Input(istream& fromStream)
{
    // Формат (3 реда):
    // <име> <ЕГН>
    // <Град> <Пощ.код> <Адрес>
    // <Факс> <МаркаGSM> <ТарифенПлан> <Тел№>
    CPerson::Input(fromStream); // ред 1
    fromStream >> m_address; // ред 2
    fromStream >> m_strFN; // ред 3, първа част
    InputGSM(fromStream); // ред 3, останалото
    return fromStream;
}

// ----- 4. Трите задължителни функции на CGSM -----
ostream& CStudent::OutputGSM(ostream& toStream) const
{
    toStream << GetModel() << " (план " << GetTariff() << ", тел. "
        << getGSMNumber() << ")";
    return toStream;
}

string CStudent::getGSMNumber() const { return m_strPhone; }

istream& CStudent::InputGSM(istream& fromStream)
{
    string strModel, strPhone;
    int nTariff = 0;
    if (fromStream >> strModel >> nTariff >> strPhone)
    {

```

```

        SetModel(strModel);
        SetTariff(nTariff);
        m_strPhone = strPhone;
    }
    return fromStream;
}

// ----- 6. Член-оператор за сравнение по тарифен план -----
// Само ЕДИН параметър: лявата страна е самият обект (this).
bool CStudent::operator>(const CStudent& studR) const
{
    return GetTariff() > studR.GetTariff();
}

// ----- 3. Оператор за четене на студент -----
istream& operator>>(istream& fromStream, CStudent& stud)
{
    return stud.Input(fromStream);
}

// ----- Оператори от упражнение 4 -----
bool operator==(const CStudent& studL, const CStudent& studR)
{ return studL.GetBirthKey() == studR.GetBirthKey(); }

bool operator!=(const CStudent& studL, const CStudent& studR)
{ return !(studL == studR); }

bool operator<(const CStudent& studL, const CStudent& studR)
{ return studL.GetBirthKey() > studR.GetBirthKey(); }

bool operator>=(const CStudent& studL, const CStudent& studR)
{ return !(studL < studR); }

void ConsoleInputEGN(CStudent& stud)
{
    cout << " Въведете ЕГН за " << stud.m_strName << ": ";
    string strEGN;
    if (!(cin >> strEGN))
    {
        cout << endl << " (няма въведени данни)" << endl;
        return;
    }
    if (stud.isValid(strEGN))
    {
        stud.m_strEGN = strEGN;
        cout << " Прието. Нова дата на раждане: " << stud.GetBirthDate() << endl;
    }
    else
        cout << " Невалидно ЕГН. Остава старото: " << stud.m_strEGN << endl;
}

// =====
// 7-8. Главна функция
// =====
int main()
{
    cout.setf(ios::fixed);
    cout.precision(2);

```

```

// ===== 1. Четене на адрес от текстов поток =====
cout << "===== 1. operator>> за CAddress =====" << endl;
{
    istringstream iss("Varna 9010 Studentska#1");
    CAddress addr;
    iss >> addr;
    cout << " прочетено: " << addr << endl;
}

// ===== 2. Четене на личност =====
cout << endl << "===== 2. operator>> за CPerson =====" << endl;
{
    istringstream iss("Ivanov 7503081239");
    CPerson pers;
    iss >> pers;
    cout << " прочетено: " << pers << endl;
}

// ===== Виртуални функции: проблемът от упражнение 2 е решен =====
cout << endl << "===== Виртуални функции =====" << endl;
CAddress addrVarna("Studentska#1", "9010", "Varna");
CStudent studDemo("Георги Стоянов", addrVarna, "7503081239", "21621301",
                  "NOKIA", 20, "0888000000");
studDemo.SetGender("мъж");
studDemo.AddGrade(5.50);

CPerson* pPerson = &studDemo; // указател към БА30В клас
cout << " през CStudent&: " << studDemo << endl;
cout << " през CPerson*: " << *pPerson << endl;
cout << " (двата реда са еднакви – това прави virtual)" << endl;

// ===== Абстрактният клас през указател CGSM* =====
cout << endl << "===== 4. Абстрактният клас CGSM =====" << endl;
CGSM* pGSM = &studDemo; // студентът Е и CGSM
cout << " през CGSM* -> OutputGSM: ";
pGSM->OutputGSM(cout) << endl;
cout << " през CGSM* -> getGSMNumber: " << pGSM->getGSMNumber() << endl;
// CGSM gsm; // грешка: абстрактен клас не може да се създаде

// ===== 8. Четене на масив от студенти от файл =====
cout << endl << "===== 8. Четене от файл Tema05.txt =====" << endl;
const int MAX_STUD = 10;
CStudent arrStud[MAX_STUD];
int nRead = 0;

fstream iFile("Tema05.txt", ios_base::in);
if (!iFile)
{
    cout << " Файлът Tema05.txt не е намерен!" << endl;
}
else
{
    while (nRead < MAX_STUD && (iFile >> arrStud[nRead]))
        nRead++;
    iFile.close();
    cout << " Прочетени " << nRead << " студента:" << endl;
    for (int i = 0; i < nRead; i++)
        cout << " " << (i + 1) << ". " << arrStud[i] << endl;
}

```

```

if (nRead >= 2)
{
    // ===== 6. Член-оператор > по тарифен план =====
    cout << endl << "===== 6. Сравнение по тарифен план (член-оператор) =====" << endl;
    for (int i = 0; i < nRead; i++)
        cout << " " << arrStud[i].GetName() << ": план "
            << arrStud[i].GetTariff() << endl;

    cout << "---- Резултати ----" << endl;
    cout << " " << arrStud[0].GetName() << " > " << arrStud[1].GetName()
        << " (по план): " << (arrStud[0] > arrStud[1] ? "да" : "не") << endl;
    cout << " " << arrStud[1].GetName() << " > " << arrStud[0].GetName()
        << " (по план): " << (arrStud[1] > arrStud[0] ? "да" : "не") << endl;

    // Двата вида сравнение върху едни и същи обекти
    cout << "---- Две различни сравнения, едни и същи обекти ----" << endl;
    cout << " по ВЪЗРАСТ (глобален <): " << arrStud[0].GetName() << " < "
        << arrStud[2].GetName() << " = "
        << (arrStud[0] < arrStud[2] ? "да" : "не") << endl;
    cout << " по ТАРИФА (член >): " << arrStud[0].GetName() << " > "
        << arrStud[2].GetName() << " = "
        << (arrStud[0] > arrStud[2] ? "да" : "не") << endl;

    // Най-скъпият тарифен план
    int nMax = 0;
    for (int i = 1; i < nRead; i++)
        if (arrStud[i] > arrStud[nMax])
            nMax = i;
    cout << " Най-висок тарифен план: " << arrStud[nMax].GetName()
        << " (" << arrStud[nMax].GetTariff() << ")" << endl;
}

// ===== Масив от указатели към базовия клас =====
cout << endl << "===== Полиморфизъм през масив от указатели =====" << endl;
CPerson persPlain("Иван Петров", "мъж", "8512034566");
CPerson* arrPtr[3] = { &persPlain, &studDemo, &arrStud[0] };
for (int i = 0; i < 3; i++)
    cout << " [" << i << "] " << *arrPtr[i] << endl;

// ===== Виртуален деструктор =====
cout << endl << "===== Виртуален деструктор =====" << endl;
CPerson* pDyn = new CStudent("Петър Колев", addrVarna, "0147222342", "21621399",
    "APPLE", 15, "0888000009");
cout << " " << *pDyn << endl;
delete pDyn; // вика ~CStudent, защото деструкторът е virtual
cout << " обектът е унищожен коректно" << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

```
===== 1. operator>> за CAddress =====
    прочетено: Varna 9010, Studentska#1
```

```
===== 2. operator>> за CPerson =====
    прочетено: Ivanov (неопределен), ЕГН: 7503081239, роден: 08.03.1975
```

```
===== Виртуални функции =====
```

```
    през CStudent&: Георги Стоянов (мъж), ЕГН: 7503081239, роден: 08.03.1975, ф.н.: 21621301,
адрес: Varna 9010, Studentska#1, GSM: NOKIA (план 20, тел. 0888000000), оценки: 5.50 | ср.
успех: 5.50
```

```
    през CPerson*: Георги Стоянов (мъж), ЕГН: 7503081239, роден: 08.03.1975, ф.н.: 21621301,
адрес: Varna 9010, Studentska#1, GSM: NOKIA (план 20, тел. 0888000000), оценки: 5.50 | ср.
успех: 5.50
```

```
    (двата реда са еднакви – това прави virtual)
```

```
===== 4. Абстрактният клас CGSM =====
```

```
    през CGSM* -> OutputGSM: NOKIA (план 20, тел. 0888000000)
```

```
    през CGSM* -> getGSMNumber: 0888000000
```

```
===== 8. Четене от файл Тема05.txt =====
```

```
    Прочетени 4 студента:
```

```
    1. Ivanov (неопределен), ЕГН: 1111111104, роден: 11.11.1911, ф.н.: 21621301, адрес: Varna
9010, Studentska#1, GSM: NOKIA (план 20, тел. 0888000000), оценки: (няма)
```

```
    2. Ivanka (неопределен), ЕГН: 1111111110, роден: 11.11.1911, ф.н.: 21621302, адрес: Varna
9010, Studentska#2, GSM: NOKIA (план 10, тел. 0888000001), оценки: (няма)
```

```
    3. Petrov (неопределен), ЕГН: 7503081239, роден: 08.03.1975, ф.н.: 21621303, адрес: Sofia
1000, Tsar_Osvoboditel#12, GSM: SAMSUNG (план 35, тел. 0888000002), оценки: (няма)
```

```
    4. Elena (неопределен), ЕГН: 0147222342, роден: 22.07.2001, ф.н.: 21621304, адрес: Burgas
8000, Aleksandrovska#5, GSM: APPLE (план 15, тел. 0888000003), оценки: (няма)
```

```
===== 6. Сравнение по тарифен план (член-оператор) =====
```

```
    Ivanov: план 20
```

```
    Ivanka: план 10
```

```
    Petrov: план 35
```

```
    Elena: план 15
```

```
--- Резултати ---
```

```
    Ivanov > Ivanka (по план): да
```

```
    Ivanka > Ivanov (по план): не
```

```
--- Две различни сравнения, едни и същи обекти ---
```

```
    по ВЪЗРАСТ (глобален <): Ivanov < Petrov = не
```

```
    по ТАРИФА (член >): Ivanov > Petrov = не
```

```
    Най-висок тарифен план: Petrov (35)
```

```
===== Полиморфизъм през масив от указатели =====
```

```
    [0] Иван Петров (мъж), ЕГН: 8512034566, роден: 03.12.1985
```

```
    [1] Георги Стоянов (мъж), ЕГН: 7503081239, роден: 08.03.1975, ф.н.: 21621301, адрес: Varna
9010, Studentska#1, GSM: NOKIA (план 20, тел. 0888000000), оценки: 5.50 | ср. успех: 5.50
```

```
    [2] Ivanov (неопределен), ЕГН: 1111111104, роден: 11.11.1911, ф.н.: 21621301, адрес: Varna
9010, Studentska#1, GSM: NOKIA (план 20, тел. 0888000000), оценки: (няма)
```

```
===== Виртуален деструктор =====
```

```
    Петър Колев (неопределен), ЕГН: 0147222342, роден: 22.07.2001, ф.н.: 21621399, адрес: Varna
9010, Studentska#1, GSM: APPLE (план 15, тел. 0888000009), оценки: (няма)
```

```
    обектът е унищожен коректно
```

Забележете реда „по ВЪЗРАСТ ... = не“. Ivanov е роден 1911, Petrov — 1975. Значи Ivanov е по-възрастен, тоест **не** е по-млад. Операторът работи правилно; резултатът само изглежда изненадващо, защото ЕГН-тата от условието кодират 1911 година.

Чести грешки

Код	Съобщение	Причина и решение
C2259	'CGSM': cannot instantiate abstract class	Опит да се създаде обект от абстрактен клас, или <code>CStudent</code> не е реализирал и трите напълно виртуални функции. Списъкът на липсващите е под съобщението.
C2804	binary 'operator >' has too many parameters	Член-операторът е написан с два параметъра. Като член приема само десния операнд.
C3892	cannot assign to a variable that is const	В <code>operator>></code> обектът е обявен като <code>const</code> . При четене <code>const</code> не се пише.
C2385	ambiguous access of 'Output'	Двата базови класа имат функция с еднакво име. Пишете изрично <code>stud.CGSM::Output(...)</code> .
C3668	method with override specifier did not override	Сигнатурата в наследника се разминава с базовата — най-често липсва <code>const</code> .
—	През <code>CPerson*</code> се вижда само личностната част	Липсва <code>virtual</code> пред <code>Output</code> в базовия клас.
—	Подменената функция не се вика	Сигнатурата се разминава (липсващ <code>const</code> , друг тип параметър). Добавете <code>override</code> , за да го улови компилаторът.
—	Теч на памет при <code>delete</code> през базов указател	Деструкторът на базовия клас не е <code>virtual</code> .
—	Прочитат се 0 студента	<code>Тема05.txt</code> не е в работната папка, или форматът се разминава — най-често интервал в името на улицата.
—	Модел и тарифа не се копират при <code>a = b;</code>	В <code>operator=</code> липсва <code>CGSM::operator=(stud);</code> .
—	<i>Stack overflow</i>	<code>CStudent::Output</code> вика себе си — липсва <code>CPerson::</code> пред извикването.

Обобщение на темата

<code>virtual</code>	Изборът на функция става по типа на обекта , не на указателя. Пише се в базовия клас.
<code>= 0</code>	Напълно виртуална: няма тяло, класът става абстрактен, наследникът е длъжен да я напише.

<code>virtual ~</code>	Задължителен, ако ще се трие през базов указател.
два базови класа	Всички конструктори, копиращият конструктор и <code>operator=</code> трябва да обслужат и двата.
оператор + виртуална	Глобалният оператор делегира на виртуална член-функция — така един оператор обслужва цяла йерархия.

Какво следва

Упражнение 6	Абстрактни класове върху масив от указатели, типови преобразувания, изключения и шаблонни класове.
Упражнения 9–13	STL: <code>vector</code> , <code>list</code> , <code>set</code> , <code>map</code> и алгоритмите над тях — готови контейнери вместо ръчните масиви и указатели.